122ME0914_Thesis - Copy.docx

National Law University, Odisha

Document Details

Submission ID
trn:oid:::29834:138670343

Submission Date
May 12, 2026, 4:47 PM GMT+5:30

Download Date
May 12, 2026, 5:46 PM GMT+5:30

File Name
122ME0914_Thesis - Copy.docx

File Size
2.4 MB

43 Pages

10,573 Words

58,684 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

17 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **17** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	repository.unja.ac.id <1%
2	Internet	acuresearchbank.acu.edu.au <1%
3	Submitted works	University of West Florida on 2024-11-24 <1%
4	Submitted works	Letterkenny Institute of Technology on 2021-09-09 <1%
5	Internet	boris.unibe.ch <1%
6	Internet	byersgillsolarfarm.co.uk <1%
7	Internet	studentsrepo.um.edu.my <1%
8	Internet	www.vmakeu.com <1%
9	Submitted works	RMIT University on 2017-09-25 <1%
10	Publication	Tian Luan, Shixiong Zhou, Lifeng Liu, Weijun Pan. "Tiny-Object Detection Based o... <1%

11 Submitted works

University of Florida on 2016-12-12 <1%

12 Internet

hdl.handle.net <1%

13 Internet

theses.ncl.ac.uk <1%

14 Submitted works

Rhodes University on 2020-02-14 <1%

15 Internet

okeanis.lib.teipir.gr <1%

16 Internet

www.linuxquestions.org <1%

Abstract

Random extrusion errors and mechanical failures plague fused deposition modeling (FDM) printing and negatively impact the integrity of finished parts, material waste, and machine damage. Quality control measures for FDM are only performed after manufacturing has completed and lack automated and closed-loop feedback. Current manual post-process inspection procedures are unable to meet the scalability demands of Industry 4.0. Therefore, this thesis work develops a real-time computer vision framework for detecting print defects during the additive manufacturing process using YOLOv26l. Over 10,219 augmented images were used to train our end-to-end model to detect five different morphological defect types and normal prints: blobs, spaghetti, stringing, under-extrusion, and warping with a total of 22,028 annotations.

Modified versions of YOLOv26l's NMS-free, end-to-end detection algorithm, C3K2 spatial blocks, and MuSGD optimizer were implemented into the network for deterministic, low latency edge inference. The network quantitative analysis proves our model can meet industry thresholds for accuracy and sensitivity with a precision of 73.63% and mAP@50 of 60.85%. Physical trials were conducted on an Ender 3 printer with deliberate defect creation. Monitoring this print occurred using a budget IP camera interfaced with a smartphone and custom Streamlit web-app with asynchronous email alerts. This work proves AI powered optical supervision can mitigate resource waste and human labor for accurate and approachable closed loop automation in desktop 3D printing.

Table of Contents

6

Table of Tables

Table of Figures

Chapter 1: Introduction

1.1 Background of Additive Manufacturing and FDM Technology

Additive manufacturing is a manufacturing process where a physical object is built layer-upon-layer directly from a computer model without material waste. Traditionally, parts are produced by cutting away material from a solid block using methods such as milling or turning. In additive manufacturing methods, successive layers of material are deposited until fabrication of the object is complete. Because layer upon layer of material are added to build objects, additive manufacturing can create shapes which would be either impossible or prohibitively expensive using traditional methods, and minimizes waste because material is only deposited where necessary.

One of the most common additive manufacturing techniques is Fused Deposition Modeling (FDM). FDM works by melting and extruding thermoplastic material onto a platform in layers. These thermoplastic materials are usually loaded into the printer on a spool and are commonly made of PLA (Polylactic Acid) or ABS (Acrylonitrile Butadiene Styrene). The material is melted down to a semi-liquid form and extruded through a nozzle, following a predetermined path. The plastic then quickly hardens once deposited, bonding to the layer below.

The first step in the FDM process is designing the object you wish to print in 3D modeling software. Once completed, the 3D model is loaded into slicing software that will cut the model up into thin layers and create a G-code file for the printer. The G-code file contains all of the necessary motions, toolpaths, extrusion rates, and layer heights for the printer to fabricate the object. The printer reads the G-code file and fabricates your object layer-by-layer by moving the extrusion head along three axes over a build platform, melting and depositing the plastic. In order to reduce printing time, a fan usually blows on the extruded material to cool it faster.

Industrial interest in FDM has increased due to its simplistic manufacturing process and accessible materials allowing for complex geometries with lightweight structures and minimal waste production. FDM is frequently used in aerospace and automotive industries, as well as rapid prototyping due to the customization options and quick turnaround times achievable without

tooling. Custom geometries, internal channels, lattice structures, lightweight parts, and reduced lead times are all possible with FDM printing.

Limitations associated with FDM printing arise from the process being built layer-by-layer. Each layer of material directly bonds to the layer below, so if there is any deformation, inconsistent extrusion, or poor adhesion that occurs in earlier layers, those errors will become pronounced throughout the rest of the build. This can result in parts that have poor dimensional accuracy and mechanical properties. If the print bed shifts during printing, the entire layer that was printed while the bed was misaligned will be shifted, and all layers printed after that will be shifted as well. This is why it is important to monitor your prints while they are running to help ensure parts are fabricated correctly.

1.2 The Shift Toward Industry 4.0 and Automated Quality Control

Industry 4.0 represents the shift towards smart, interconnected, cyber-physical manufacturing. Production facilities under this new age industrial revolution are often equipped with IoT sensors, utilize cloud computing/big data, and leverage AI solutions to collect feedback from physical production lines and then use this information to improve said production lines. As additive manufacturing can fall under this umbrella as well, many technologies native to AM are developed with IoT and realtime control in mind. Fabrication via FDM is done layer by layer in a digital native fashion which allows for easy integration of sensors and controllers to allow for realtime data capture and actuation.

Prior to smart technologies being widely used for quality assurance, QA was most often done by having a human watch the print and visually inspect the part as it was printed at certain intervals. The human operator would have to watch the print nozzle and the surface of the part to ensure there were no surface defects, layer shifting, or extrusion errors. Although this method works well for low volume or R&D settings, there are limitations to using a human operator for quality assurance. For one, humans have different skill levels so there may be high variation in the QA results when done by different operators. Also, one human cannot watch the print process for long

periods of time without getting tired and if they do become tired, there is a high chance they will miss defects. In addition, often there are multiple printers printing for hours or even days which makes it difficult and costly to have humans watch every printer.

Lastly, there is only so small of a defect that a human can physically see. During the initial layers of a print there will often be small defects that a human may not notice but the defect can grow larger as the layer height gets bigger. Because of this, manual inspection is said to not be scalable to meet industrial needs.

Because of this need for smart quality assurance, one of the first steps towards smart manufacturing has been the implementation of AI and computer vision. Cameras or other high resolution optical sensors can be placed inside of the printer's enclosure which allows for photos of the part's surface to be taken after each layer is complete. In conjunction with a deep learning inference model, these images can be used to detect surface defects, dimensions, and extrusion errors.

Vision processing allows for defects to be detected as they happen at the layer they occur at instead of after the part is printed and goes through destructive testing or a coordinate measuring machine. If a defect is found, the print can be paused or tweaked to help prevent wasted materials and failed prints all without human supervision. Having a closed looped quality assurance process is a large step towards smart AM that follows the principles of Industry 4.0.

1.3 Problem Statement

Although Fused Deposition Modeling has come a long way as a technology, random mechanical failures and extrusion-related errors have remained a serious threat to repeatability and reliability of the process. Due to the additive nature of the FDM process, any disruptions in extrusion dynamics, thermal dynamics, or adhesion to the build platform affects layer-wise deposition of the printed geometry. These disruptions caused by any unforeseen error/sources of instability will propagate through later layers if left unchecked.

Some defects like blobs and zits (due to pressure variation within the extruder), or strings that form between features when molten plastic is unintentionally deposited as the extruder moves between

features, have an obvious impact on the dimensional accuracy of the printed part. Some defects such as under-extrusion leads to poor layer bonding, poor surface finish and gaps within the deposited layers, while warping compromises part shape and adhesion to the build platform. Dimensions are further lost if the printed part detaches from the build platform or collapses during printing. Perhaps the worst type of failure which can occur during an FDM print is known as the spaghetti failure. As shown in Figure 1g, when the printer continues extruding unfused plastic into mid-air after a part of the geometry has detached from the build platform or collapsed during printing. Since FDM is an additive process, once a defect occurs, it usually doesn't fail-forward; thus, printing will continue until the entire job is completed unless detected by some means of intervention. Additionally, when large enough, the spaghetti formed from such a failure can cause the print head to collide with previously deposited material or even stick to the nozzle, causing further damage to the printer.

Therefore, a lack of real-time monitoring is the major problem that this project aims to solve. As of now, unless enabled by the user, most printers do not monitor the print as it is being printed and any errors that occur are noticed only after the completion of the print job when one simply looks at the print. By that time, the printer would have used the entire time allocated for the print as well as the entirety of the filament that would've been used had the print been successful. This makes FDM printing ill-suited for applications in which printed parts are required to function reliably as mechanical parts. Therefore, there is an immediate need to develop a process that detects these failures as they occur and stops the printer, saving time, filament, and prevents damage to the printer.

1.4 Motivation of the Research

This project aims to increase the reliability, uptime, and material efficiency of FDM-style additive manufacturing through the use of AI-powered computer vision software. Fused deposition modeling is quickly becoming one of the more utilized forms of additive manufacturing across industry, prototyping, and even small-scale manufacturing applications. However, the field still faces the problem of lacking an automated quality control solution.

A real time print failure detection system allows for objective, consistent monitoring of print quality during the entirety of the build process and mitigates one of the biggest downsides to traditional manual post-mortem quality inspection.

Motivation #1: Prevent Complete Print Failures and Damage to Printer Hardware

Printing defects like spaghetti fails, blobs, stringing, under extrusion, and warping have the potential to completely ruin a print. But when left unchecked, the printer will continue to indiscriminately lay down filament on top of a failed print. If left to run its course, not only will all of the filament allocated for that particular layer be wasted but extruded plastic from the spaghetti effect or severe crashes can build up at the nozzle and hot end. This can cause clogs as well as heat damage to the printer. Automatic detection of defects when they first occur at the layer, they happen will allow the print to be stopped before damage to the printer occurs or before the part is completely ruined. By stopping the print early, you save the remainder of your filament as well as your printer.

Motivation #2: Reduce Wasted Filament and Machine Time

Running a failed print with no method of monitoring it in real time can lead to hundreds of dollars in wasted filament and machine time. Instead of catching the failure as soon as it occurs, the printer typically prints until the estimated amount of filament for that print has been used. Not only does this waste the filament but all of the time that was allotted for the print to complete on the machine. Automatic detection would allow the print to be stopped when the defect is first recognized, saving much of the filament that would otherwise be used. Minimizing failed print waste also means less machine time is wasted printing something that won't work. This can become extremely useful if you have multiple machines printing continuously for long periods of time unattended.

Motivation #3: Increase Inspection Consistency Through Full Automation

Traditional quality control of FDM prints comes in the form of manual inspection which is very inconsistent. Automatic detection and inspection of prints remove all of the subjectivity that comes with manual inspection. A computer will not become tired of watching prints or miss layers due to boredom. A computer vision model can look at every layer of a print with the same sensitivity

and consistency from a print that takes an hour to one that takes twelve hours.

1.5 Objectives of the Project

Project goals were initially laid out as technical milestones that further segmented the overall development of a real-time AI-based defect detector for FDM additive manufacturing processes into stages with measurable deliverables:

Goal Number 1: Build dataset

Motivation: Bridge dataset gap outlined in problem statement

Description: Gathered and unified data samples consisting of pre-loaded annotated images sourced from Roboflow datasets with the intention of building a well-rounded, visually heterogeneous training dataset of five FDM processing defects which include: blobs, spaghetti printing, stringing, under-extrusion, and warping. Images were sourced from multiple datasets to provide large class variation as well as minimize dataset specific biases to ensure model generalizability across different printers, lighting schemes, filaments, etc.

Goal Number 2: Develop detector

Motivation: Compile model from knowledge gained

Description: Train lightweight model from scratch using systematic hyperparameter tuning of YOLO26l object detection architecture. Training was tasked with identifying where each printing defect existed in each frame of video while also classifying what type of defect was occurring. This goal served as a conclusive checkpoint to demonstrate model knowledge gained through training.

Goal Number 3: Test performance

Motivation: Ensure model can operate at speeds required for real-time analysis

Description: Quantified model efficacy and operation speed through common object detection benchmark metrics such as precision/recall, mAP, and frames per second. This step was crucial to prove that the model could detect defects at a rate that analyzed the video stream of the printer in real time.

Goal Number 4: Inference on printer webcam feed

Motivation: Validate end-to-end project functionality

Description: Programmed and executed a Python script to run inference on webcam footage of the FDM printer in real time to showcase autonomous defect detection. This script was able to notify the user and send an email alert when a defect started occurring. Program operated independently while the print was running to exhibit proof of concept. Fully implemented program to interface with printer and stop the print job upon defect detection via G-code commands is left to future development.

2

Chapter 2: Literature Review

2.1 The Physics and Mechanics of FDM Defects

It is important to understand the science behind why these defects occur in order to confidently address and prevent FDM issues. Each failure mode happens for a different reason related to the physics of extrusion.

Warping is defined by the thermal expansion coefficient of the plastic. When the material first extrudes, it is cooled from its glass transition temperature all the way down to room temperature. Due to this difference in cooling, the corners of the object develop large amounts of residual stress. If the part develops enough thermal stress that the forces pushing the corners upwards are greater than the forces keeping the corners stuck to the print bed, your corners will lift up and the print will fail.

Blobs happen when large amounts of excess plastic depositions on the outside of your part. This defect is mainly caused by extruding the plastic at too high of a temperature or not having enough cooling to properly solidify the plastic when it is deposited. This results in the print sag due to gravity and local heat, causing large defects.

Stringing happens when there is not enough retraction when the nozzle moves across open space. When the nozzle needs to move from one place to another on the print without extruding, sometimes there is still too much pressure in the hot end melt chamber and the plastic leaks out of the nozzle. This leaves a small string of plastic between gaps in your print.

Under extrusion looks like holes, gaps in the layers, or a “spongy” outer appearance. This happens when the printer does not extrude enough filament through the nozzle. There can be many reasons for not extruding enough plastic, but the most common are partially clogged nozzles, having the incorrect flow rate percentage in your slicer, or loose filament in the extruder gear.

Spaghetti is a large blob of failed plastic that lacks anything to build on. This occurs when the object falls off the print bed or if you try to print over a gap without enough support underneath. When this happens, the nozzle continues to extrude into nothing and you are left with a useless print.

2.2 Impact of Defects on Mechanical Integrity

The fundamental mechanics of Fused Deposition Modeling (FDM) dictate that the layer-by-layer manufacturing process introduces inherent anisotropic behavior to the printed parts. This anisotropy means that the mechanical properties of a 3D printed component, including its tensile strength, yield behavior, and elastic modulus, vary significantly depending on the orientation of the applied load relative to the print direction. Because of this directional dependency, structural integrity is highly sensitive to the quality of the inter-layer and intra-layer bonding, making FDM parts particularly vulnerable to mechanical degradation when process-induced defects occur.

The presence of extrusion anomalies, particularly gaps caused by under-extrusion, drastically compromises the load-bearing capabilities of a manufactured part. Academic literature investigating the specific impact of these voids on mechanical performance has quantified the severity of this degradation. Research demonstrates that when missing extrudates or gaps occur perpendicular to the transverse loading direction, the compromised parts exhibit a 20.5% reduction in overall tensile strength compared to baseline defect-free models.

Furthermore, these structural voids negatively impact the material's stiffness and ductility. The same empirical studies reveal a corresponding 9.6% reduction in the elastic modulus and an 11.5% reduction in the failure strain due to the presence of these under-extrusion gaps. These severe mechanical degradations explicitly highlight the vulnerability of FDM components to process variations and underscore the unreliability of visually adequate but internally defective prints. Consequently, this quantitative evidence necessitates the implementation of an in-situ monitoring system to actively detect extrusion anomalies and prevent mechanically compromised parts from entering functional engineering applications.

2.3 In-Situ Monitoring Modalities in Additive Manufacturing

Additive manufacturing processes often require online or in-situ monitoring, since defects occur gradually layer-by-layer and cannot be fixed after the process has begun. There are various monitoring techniques that have been explored in literature to watch machine and print characteristics throughout fabrication. Each technique presents its own tradeoffs when considering what kinds of defects are easily detectable, where the defect is located on the print, and if monitoring can be performed in real-time.

Acoustic monitoring has been explored that allows microphones and signal-processing algorithms to listen to sounds from the printer. Different sounds correspond to abnormal printer movements, unstable extrusion, or structural failure. Algorithms have been derived to detect acoustic anomalies and they can run efficiently in real-time. Although acoustic monitoring is inexpensive and non-invasive, noise pollution in industrial or laboratory environments with other machinery can interfere with real-time detection. Additionally, the source of a defect cannot be located using acoustic monitoring. Because the coordinate of the defect is unknown, acoustic anomaly detection cannot detect if a stringing event or blob occurred at a certain point on a geometry. Nor will it be able to precisely detect the region of under-extrusion.

Thermal monitoring is another technique often used to watch print failures. One implementation of thermal monitoring includes infrared cameras or thermal sensors. Since FDM is ruled by transient thermal processes (heat transfer, thermal gradients, cooling rate, thermal shrinkage), it is useful to understand these thermal effects that cause warping, residual stresses and weak layer bonding. Therefore, many failure modes that are caused by thermal inconsistencies can be detected and analyzed with thermal monitoring. However, certain failures will not be thermal-specific. Print defects such as spaghetti failures, pile ups, and slight surface blemishes will not have unique enough thermal signatures to classify.

Optical monitoring using a camera is different from acoustic and thermal monitoring because it allows for the direct and explicit visualization of the printed layers and geometry as they are deposited. Computer vision allows the printer to watch for any morphological inconsistencies that arise during printing: stringing, blobs, warping that causes layers to detach from the build platform,

under extrusion, and catastrophic failure (aka. spaghetti failure). Since cameras are detecting the failures in real time, the system also knows where the failure occurred and how much of the geometry is affected by that failure. Unlike thermal cameras, optical systems are able to explicitly detect failures that occur as surface-level defects or extrusion-based defects. Therefore, optical monitoring paired with computer vision was chosen as an ideal modality for in-situ detection of print defects because the printer is able to watch for any unusual activity on the surface of the print as it is being printed.

2.4 Evolution of YOLO Architectures for Object Detection

YOLO detectors were created with the goal of replacing detection frameworks that used pipeline-based approaches with a single unified real-time prediction system. The dominant method of object detection before the introduction of YOLO models was region proposal-based detection. Systems like R-CNN and its successors proposed candidate regions and fed them through detection pipelines individually to classify objects within candidate boxes. While detection accuracy was robust with these methods, speed was not which limited their implementation in industrial inspection contexts.

YOLO unified the detection process by predicting bounding boxes and class probabilities for those boxes in a single regression problem. The image was split into an S×S grid and each grid cell was responsible for predicting objects whose center fell within that cell. While these initial models lacked finer-grained localization ability and struggled with small objects within scenes, they were able to achieve real-time performance which proved impactful for industrial detection tasks.

Improvements to localization ability and overall robustness were made in the following iterations of YOLO. By including batch normalization layers, implementing anchor boxes, and optimizing the Darknet backbone, YOLOv2 was able to converge better and more quickly during training while also improving localization accuracy. YOLOv3 added an even deeper version of Darknet, while predicting bounding boxes at 3 different scales. Together, these improvements allowed YOLO to detect small and medium-sized objects with greater accuracy.

Later models would continue to improve both accuracy and inference speed for easier deployment. YOLOv8 improved upon detection through the use of decoupled heads and anchor-free detection, making box prediction easier by removing the need for explicit label assignment while also decreasing training divergence. Since then, models have sought to improve the lightweight nature of their predecessors. YOLOv11 removed previous cross-convolution modules and replaced them with smaller C3k2 blocks to increase feature network density while decreasing redundant bottleneck layers.

YOLOv26 can be thought of as continuation of this legacy. While architecture size and complexity continues to grow, this version seeks to prioritize efficient inference and smooth deployment within real-time contexts. It accomplishes this by utilizing an end-to-end NMS-free detection scheme and removing the Distribution Focal Loss from its detection head. By removing these two elements, latency spikes within the model are reduced allowing the network to operate in a more deterministic and consistent manner. This consistency allows YOLOv26 models to be faster and more computationally efficient on edge devices, allowing for increased flexibility during deployment over more complex models that may offer only marginal increases in accuracy.

Due to this, YOLOv26l was chosen for this project. In defect detection applications, speed is of the essence. This model was chosen due to its compatibility with real-time systems and overall robustness. As mentioned before, an in-situ monitoring system requires reliable detections over extended periods of time while abiding by the constraints of a physical system. An end-to-end, low-latency optimized detector allows for these detections to occur as fast as possible ensuring alerts can be raised before too many defective prints are produced or too much material is wasted.

Chapter 3: Theoretical Framework

3.1 Defect Typology and Morphological Signatures

For an object detection algorithm to be able to classify images as containing certain additive manufacturing defects, each defect class must possess repeatable visual features to learn. Every defect in FDM has its own morphology that sets it apart from the appearance of normally extruded layers. This unique morphology can include shape, texture, size, height, location, and other spatial information which can be observed by the human eye. What the model ultimately learns to identify are these features extracted from the ground truth label images that have been provided to it during training. Thus, we identify here what the human eye recognizes about the morphology of each individual defect class.

Blobs have a morphology that can be described as spherical or irregular deposits of excess material on the layer's surface or the part's sidewalls. They are dense in morphology and appear as isolated defects protruding from the smooth and continuous linear shapes of surrounding good-deposited layers. Blobs tend to visually stand out from well deposited layers due to their uneven shape, increased height, and interruption of layer beds. Therefore, blobs are spatially salient with surface features that make them easy to identify.

Spaghetti is perhaps one of the most visually noisy of FDM's defect classes. A bird's nest of intertwined melted plastic wisps that vary in shape and size compose what appears to be a large lump of deformity. There is no structural rigidity, organized layer shape, or consistent size to the morphology of spaghetti. What is left in its place is a high texture value, random layer curvature, and unpredictable filament trajectories that divert greatly from normal layer deposition.

Strings are extremely thin strands of filament that connect isolated areas of the part. Occurring typically between travel moves of the nozzle, strings appear as transparent horizontal lines across gaps in the model. Although they are very low in pixel value and thin in size, their linear shape is continuous and detailed enough to create a form of ordered noise. Object detection understands stringing as a defect class that is defined by less volume and more by unique filament connectors.

Cavities within what should be solid surfaces are what define under extrusion. The perimeter walls or infill will have gaps in them or appear sponge-like rather than dense and solid. Material will often be missing in sections where the extruder should have been depositing along the toolpath. With missing filament, the surface of the layers appears incomplete or weak and can sometimes take on a patterned texture. Object detection can identify under extrusion by a lack of morphology where there should be some.

Warped prints are perhaps one of the easiest defects to spot because the bottom corners or edges of the print lift away from the print bed at an angle rather than sitting flat. As the object should normally lay perfectly perpendicular to the build plate, warping creates a change in the angle of the bottom edges and lifts the print away from the bed. Since this deformation changes the overall shape of the object in relation to the print bed, warped prints have unique morphology that can be detected.

Each of these five classes has very unique morphology that allows it to be recognized by an object detection model. Whether it be through differences in shape, size, height, or another feature, there is enough of a visual difference between defect classes to allow the model to learn the ground truth labels. The model is not looking for defects, but rather high frequency shapes and surface features that are unique to each type of printing failure.

3.2 Deep Dive into the YOLOv26m Architecture

YOLOv26 contains the backbone of our defect detection network. At a high level, it ingests an image and spits out object detections. As shown in figure it can be divided into three components: Backbone, Neck and Detection Head. This three-stage pipeline is fairly standard across contemporary single-stage detectors. Moving down the pipeline, each stage extracts visual features from the image and refines them for object prediction. In YOLOv26, we extract visual features relevant to FDM defects and perform semantic segmentation to detect these defects.

YOLOv26 Backbone consists of the main network module. We adopt a Cross-Stage Partial Network inspired (CSP-)Darknet structure. Darknet sequentially extracts deep features from shallow ones while encoding low-level information like edges, corners into higher-level concepts related to defects. To achieve this, YOLOv26 uses C3K2 blocks throughout backbone network. Darknet also uses Convolution layers with 3×3 kernels. So, we may describe the C3K2 blocks as Cross Stage Partial blocks with 3×3 kernels, which improves the local perception of the model.

Detecting defects like stringing/de-stringing, surface defects and extra-short under extrusion require local perception as they are less frequent than other defects and may get filtered out while extracting high-level features. Here, we choose to use C3K2 blocks instead of regular ones because their effective reception kernel is wider. As seen in Fig.3(b), along with C3K2 blocks, YOLOv26 backbone also has an improved Spatial Pyramid Pooling Fast (SPPF) module with shortcut connected to it. The SPPF module mines low-level features with larger field-of-view while retaining spatial information about defects. Its output is concatenated with the backbone network's output feature map. The shortcut essentially connects the input directly to the last layer which smoothen's the error gradients, allowing us to train the network faster and prevents the gradients from dying in lower layers. On an intuitive level, this means that the model can learn better features for detecting defects without worrying about network stability.

YOLOv26 Neck constitutes a Path Aggregation Network (PAN) fused with a Feature Pyramid Network (FPN). As shown in figure, this design choice is motivated by the fact that defects can be big as well as small. Large mess of spaghetti-like off-bed material easily takes up more of the frame while stringing might just look like very thin hair-like strands between two parts of the model. FPN merges semantically strong deep features with shallow weak features with rich spatial information which allows YOLOv26 to detect both big and small objects. PAN then goes on to merge these multi-scale features maps from various stages of the backbone. Therefore, YOLOv26 neck acts as a multitasking hub that aggregates and rearranges features for the detection head to interpret.

YOLOv26 also implements C2PSA blocks (Center-sensitive Position Sensitive Attention) into its neck. In figure, we see how C2PSA blocks re-contextualize local defects with their surroundings.

Position Attention allows the network to focus on long-range interactions giving it some sense of where defects are likely to occur on the print. This is important for additive manufacturing defects as they need to be contextualized with the part. An extra layer of plastic spat out by the printer is likely to be detected as a defect if it were to appear out-of-context. But if it is on the edge of the model, it can be seen as gravy. Attention based feature fusion allows the network to better understand the situation around a potential defect.

YOLOv26 Detection head and Inference strategy. Traditional detectors like Retina Net, YOLOv5 etc., perform Non-Maximum Suppression (NMS) on the network's predictions to remove duplicates and predict the final bounding boxes and classes. These bounding boxes are called anchors and are usually over abundant during prediction. NMS though effective is an extra step in inference which YOLOv26 removes by being able to predict the final detections directly. The authors made YOLOv26 learn this behavior by using a dual-head training scheme. As shown in figure, a one-to-many head and one-to-one head work together during the training process where the one-to-many head acts as a teacher providing richer supervision. During inference YOLOv26 is capable of NMS-free, inference detecting defects in an end-to-end fashion. The advantages of this approach are threefold. One, since there is no NMS post-processing step, whole inference pipeline becomes simpler to understand and deploy. Two, the inference time latency varies less because there are less operators. Three, and more important for the task at hand, since inference is deterministic, we can guarantee real-time performance on edge devices. In the context of a print job, this would mean that the system can alert the operators of a defect with minimal delay. Allowing them to stop the print before more defects occur or worse, the defect leads to failure of the print job.

YOLOv26's Loss Function design choices also align with the goal of streamlined, easy-to-deploy learning strategies. Distribution Focal Loss has been discarded from the architecture. While the inclusion of DFL benefitted localisation refinements in past YOLO models, it added unnecessary complexity to the architecture and resulted in headaches when trying to deploy the model to edge devices with limited power. The training procedure now instead uses Progressive Loss Balancing (PLB) and Small-Target-Aware Label Assignment to better control the contribution of each positive sample during optimization. Both of these automatically adjust the weight of small and/or

visually feeble targets during training to ensure they are given enough supervision. This characteristic becomes crucial to prevent defects with small support areas such as stringing and local under-extrusion from being ignored by the model.

Additionally, we use MuSGD as our optimizer of choice. It is described as a hybrid between traditional Stochastic Gradient Descent and the fictional Muon's method of rapidly reaching convergence. MuSGD promises to increase the rate of convergence while maintaining stability during large training durations. This allows for faster training on datasets with significant class morphology differences. Our use case benefits from this because we need the model to detect both large, obvious failures like spaghetti as well as light, small defects like stringing in the same model. Faster convergence means we can reach our desired accuracy without worrying about overtraining.

The Backbone, Neck, and Detection Head of YOLOv26 work in tandem to provide accurate predictions for the unique visuals of AM defects. The backbone extracts low-level details as well as semantic information. The neck combines feature maps of various sizes while also adding in context. The final detection layer then predicts bounding boxes and class probabilities in an end-to-end fashion, skipping NMS altogether. Faster optimization combined with a simpler loss function makes YOLOv26 ideal at detecting FDM print defects in real-time during the printing process.

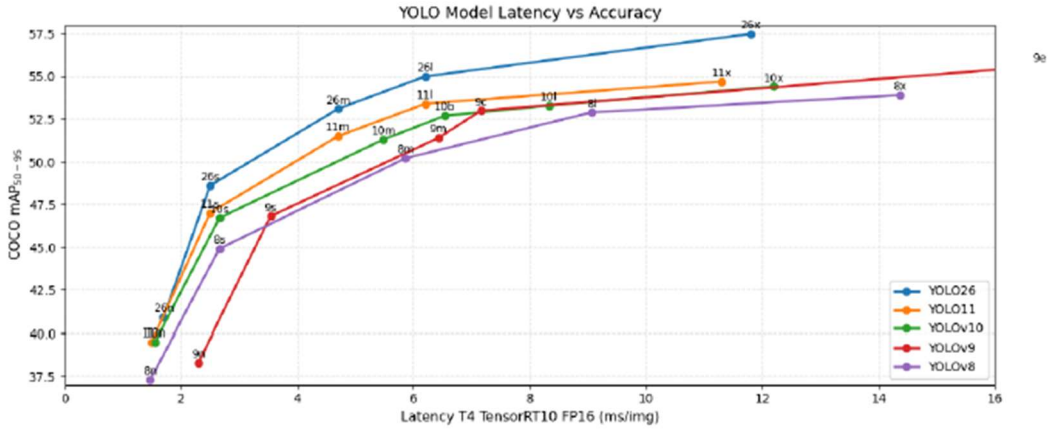
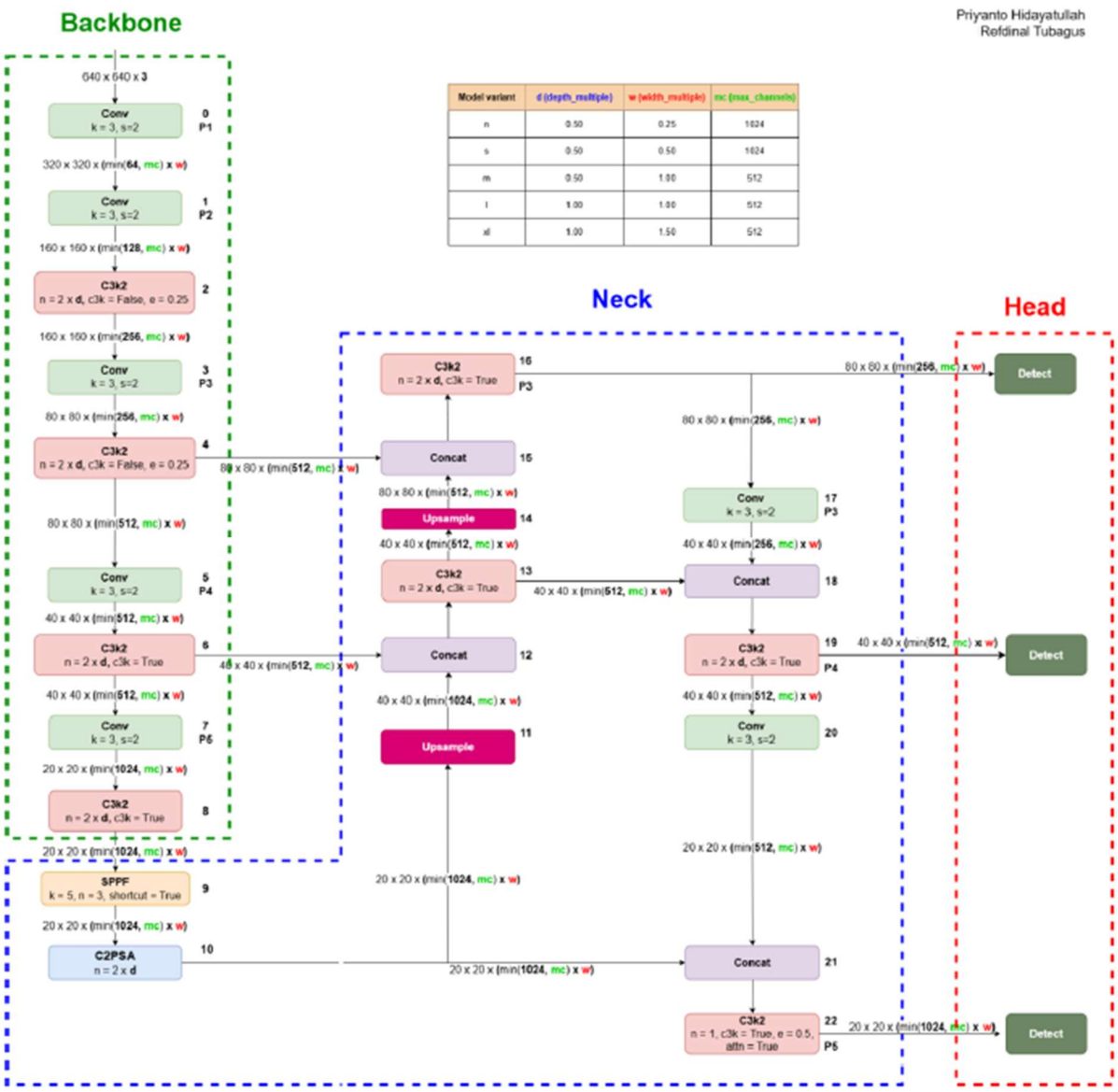


Figure 1 YOLO Performance Comparison



Model variant	d (depth_multiple)	w (width_multiple)	mc (max_channels)
n	0.50	0.25	1024
s	0.50	0.50	1024
m	0.50	1.00	512
l	1.00	1.00	512
xl	1.00	1.50	512

Figure 2 YOLOv26 Architecture Diagram

Chapter 4: Methodology

4.1 FDM Defect Typology and Dataset Aggregation Fundamentals

As with any supervised deep learning approach, the success of this model depends heavily on the topology, balance, and fidelity of the data it was trained on. To guarantee that our network had enough visual variety to generalize to new industrial conditions, our dataset was crowdsourced from several collections on Roboflow. Images from different FDM machines, lighting setups, and filaments were mixed together to create our initial dataset of 5,874 images from "scratch. "To ensure that our model could easily identify anomalies in the structure of printed parts, we made sure that every image in our dataset was densely annotated.

Our dataset contains 22,028 bounding box annotations across all five classes of topological failures with an average of 3.8 defects per image. The topologies and thermal relationships of these five target classes are detailed below and describe the most common mechanical failures that occur in FDM's layer-upon-layer additive process. Blobs are malformed, bulbous piles of excess plastic. Blobs are dense regions of plastic that form when your hot end is too hot for your chosen thermoplastic, or if your part-cooling fans aren't working well enough to cool down the printed strand fast enough. The plastic droops, leaving large deposits on what should be very precise linear paths. Next to spaghetti, blobs are the furthest thing from a fatal error. However, they can ruin the aesthetic of your print. Spaghetti is a severe failure that shows up as random swirls of misshapen, unfused plastic. Spaghetti usually occurs when a print fails completely and separates from the print bed or when you ask your slicer to extrude across open space with no support.

Mathematically speaking, your print has lost its shape retention entirely and will have a high standard deviation of visual texture. Strings are hairline-width trails of transparent plastic connecting voids in your print. Strings are almost directly correlated to poor retraction settings and too much pressure in your nozzle when it moves between sections that don't require extrusion. Due to their low-area coverage, they can be some of the easiest artifacts to spot. Under-extrusion can appear as gaps in your print, missing contours, or a sponge-like texture on the outside walls or inside fill of your print. Under-extrusion can be caused by a partially clogged nozzle, an incorrect flow rate multiplier in your slicer, or your extruder gear slipping. Physically, under-extrusion

means you aren't extruding enough material your printer isn't putting out enough plastic to match what your slicer thinks it should be. Warping appears as the corners of your print lifting up and away from the print bed. Warping is entirely dependent on the plastic you're using. All plastic has a thermal expansion rate that causes it to contract as it cools down. As your extruded strand hits the print bed, it will cool from its extrusion temperature down to room temperature. As this plastic cools at a rapid rate, warping occurs when the stress from the contraction overpowers the adhesion of your first layer to your print bed.

Training our model with Null Data Points We decided to include 1,551 images with no defects to train our model on what a properly printed layer should look like. When training our model with these examples of true whitespace, we help the model learn the balance and typical mathematical patterns of convex polygons that make up a healthy printing job. We want to heavily penalize our model for predicting false boxes within these simple geometries. Null was the most common class of images in our dataset. In the real world, this isn't necessarily a bad thing! The likelihood of encountering each of these defects represents the true prior probability of extrusion failures you would encounter at large. We found 23 defective images with no labels and removed them from our dataset as they would throw off our loss during training. Image size on our dataset averages around 0.41 MPix, with a physical range of 0.01 MPix to 49.94 MPix. The images come closer to a 640 * 640 ratio.

Table 1 FDM Defect Dataset Distribution

Morphological Class Name	Physical Defect Description	Total Annotation Count	Industrial Relative Frequency
Blobs	Bulbous excess material accumulation on surface perimeters.	13,206	High
Spaghetti	Tangled, unbonded filament from complete structural adhesion loss.	4,994	High
Stringing	Fine oozing and cobwebbing across gaps during non-extruding travel moves.	3,979	Medium
Under-Extrusion	Porous, sponge-like gaps resulting from insufficient volumetric flow rates.	1,055	Low
Warping	Orthogonal detachment of base corners due to severe thermal contraction.	794	Low

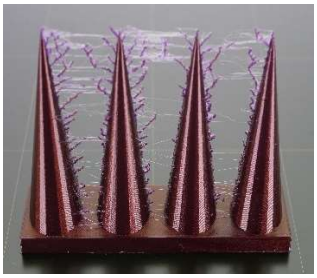


Figure 5 Stringing Defect

Figure 4 Warping Defect

Figure 3 Spaghetti Defect

Figure 6 Under-Extrusion Defect

Figure 7 Blobs Defect

4.2 Optical Preprocessing and Mathematical Augmentation Pipelines

Before feeding our dataset into the training script for YOLOv26l, we performed several preprocessing and augmentation techniques to standardize the coordinate system and increase diversity in our dataset. From our original 5,874 image count, we split our data cleanly so that no data leaks into our validation set from our training set. We allocated 4,347 images for training (74.94% of data) and cleanly set aside 1,527 images for validation (25.06%) so that we had a strict baseline to compare to that the model has never seen during backpropagation. Additionally, we performed these preprocessing steps to both the training set and validation set.

We first performed Auto-Orient which standardized all EXIF orientation data that some cameras may create, allowing for uniform coordinate systems between images. We then resized all images to a shape of 640×640. We chose to “Fit with reflected edges” when resizing our images. This prevents the aspect ratio of the printer and geometry from changing, while also not introducing black pixels on the edges of the images. Artificial black edges can negatively affect training as the CNN will learn these edges as major features. To reduce risk of overfitting and introduce some of the wild variability that exists in real-world manufacturing, we performed the following augmentation techniques on the training set only.

We did not augment the validation set in order to prevent any chance of data leaking and to have a fair ground truth to test against that is representative of actual camera conditions. The following augmentations we chose mimic some real-world conditions that occur on a factory floor: A Horizontal Flip. We horizontally flipped images at chance to simulate cameras that may be mounted backwards or mirrored on different printer gantries. This will help the CNN learn to detect holes regardless of how the camera is mounted. Rotation. We rotated our images between -10 and +10. Small rotation ranges will simulate small errors in how the camera is mounted. Large rotations will simulate high frequency jitter that occurs when the FDM printer is actively printing, as the print head and motors are rapidly accelerating and decelerating. Brightness.

We adjusted our brightness by a percent between -20% and 20%. Factory floors, especially ones with many workers moving around can experience abrupt changes in lighting conditions. Shadows may be introduced or removed. This augmentation will help our model be more robust to unknown thermal changes. Brightness could simulate some of this. Gaussian Blur. We blurred our images with a Gaussian filter of up to 1 pixel. The FDM print head typically moves so fast that there can be camera noise introduced from either a out of focus lens or general motion blur Noise. We added noise to our images which can randomly alter up to 1.53% of pixels. Noise can simulate noise that is introduced from cheap industrial webcams that are typically used in these low-light conditions. As we apply 2 augmentations to every training image, we have effectively doubled our original training dataset size. Our original training set of 4,347 images grows to 8,694 augmented training images. With 1,527 validation images, our entire dataset now contains 10,219 images.

4.3 Cloud Computing Infrastructure and Training Pipeline Orchestration

Given the substantial 55.0 million parameter footprint and the 286.2 GFLOP computational requirement of the YOLOv26l architecture, training operations were completely offloaded to cloud computing infrastructures to bypass the severe hardware limitations of local workstation machines. The extended training sequence was executed via the Kaggle computing platform, specifically leveraging the highly parallelized computational capabilities and expansive VRAM of the NVIDIA P100/T4 GPU accelerators. The 10,219-image dataset, fully processed, resized, and

augmented, was exported from the Roboflow environment formatted precisely to the YOLO PyTorch specification. To completely override previous operational configurations and maximize the theoretical accuracy boundary of the large architecture, the training configuration was explicitly extended to 96 total epochs. This extended temporal duration allows the model to leverage the MuSGD optimizer's capacity for deep, stable convergence across complex parameter landscapes without succumbing to early stagnation.

The programmatic implementation of the model initialization and the training pipeline was engineered utilizing the Ultralytics framework API, as explicitly documented below:

```
from roboflow import Roboflow

rf = Roboflow(api_key="my-api-key")
project = rf.workspace("ml-ssuvd").project("3d-printing-defect-detection-evopu")
version = project.version(2)
dataset = version.download("yolo26")

from ultralytics import YOLO

model = YOLO("yolo26l.pt")

results = model.train(
    data=f"/kaggle/working/3D-Printing-Defect-Detection-2",
    epochs=30,
    imgsz=640,
    batch=32,
    device="0,1",
    patience=5,
    cache=False,
    save_period=10,
    project="/kaggle/working/training",
    name="model_V1"
)
```

Hyperparameters were chosen to help stabilize the gradient and increase GPU memory efficiency. Image sizes were fixed at 640 to align with the dataset transforms (which were computed once beforehand), instead of dynamically resizing on each iteration. Batch size was fixed at 32 to saturate the VRAM of NVIDIA P100/T4 GPUs while having a sufficiently large sample size to

calculate accurate gradients each step. Early stopping patience was set to 5 epochs, essentially meaning if the validation loss did not improve for ten epochs the training would automatically stop preventing any possibility of this 55 million parameter network from overfitting to the training data.

4.4 Experimental Mechanics of Controlled Defect Induction

Testing against static validation data points alone is EXTREMELY inadequate without some sort of physical validation testing under real-world conditions. So in order to verify whether or not our YOLOv26l model would hold up in a real-world industrial environment we induced a very controlled experiment physically. We used our Ender 3 FDM 3D printer as our physical unit of testing. Instead of waiting around for hours for a chance defect to occur naturally, we altered the machine, kinematic, and thermal settings of the printer to FORCE immediate creation of certain physical defects. One of the main defect types we wanted to test for was “Stringing”. As we’ve learned before, stringing has the smallest physical area and is see-through making it the hardest physical object for our computer vision model to detect out of all 5 classes. Stringing is created when melted plastic that does not follow the traditional fluid dynamics act up under certain thermal and pressure conditions. We induced this by editing a custom profile through the typical Cura software and altered four physical variables which we input into the G-code file.

Table 2 Modified Slicer Settings

Parameter	Default Configuration	Modified Configuration	Physical Justification for Defect Induction
Nozzle Temperature	200°C	225°C	Drastically reduces the dynamic viscosity of the polymer. Maintaining the filament at this super-heated, highly fluid state causes severe, spontaneous oozing during non-extruding travel moves.
Retraction	45 mm	0 mm (Off)	Completely neutralizes the extruder gear's mechanical pullback. Without negative pressure retraction, the residual internal pressure in the melt zone forces the fluid filament out of the nozzle continuously.
Travel Speed	50 mm/s	40 mm/s	A slower spatial translation across non-extruding spaces provides the molten matrix more time to drip, sag, and bridge the empty structural gaps between the printed geometries.
Cooling Fan	100%	30%	Severe reduction in convective airflow prevents the rapid, required glass transition of the polymer. This keeps the stringing wisps highly fluid and elongated, maximizing the visible artificing.

Corrupted G-code customized to produce the exact errors was written to a regular SD card and inserted into the Ender-3. By forcing the printer into perpetual printing, it created a baseline for on-the-fly testing.

Figure 8 Ender 3 Printer

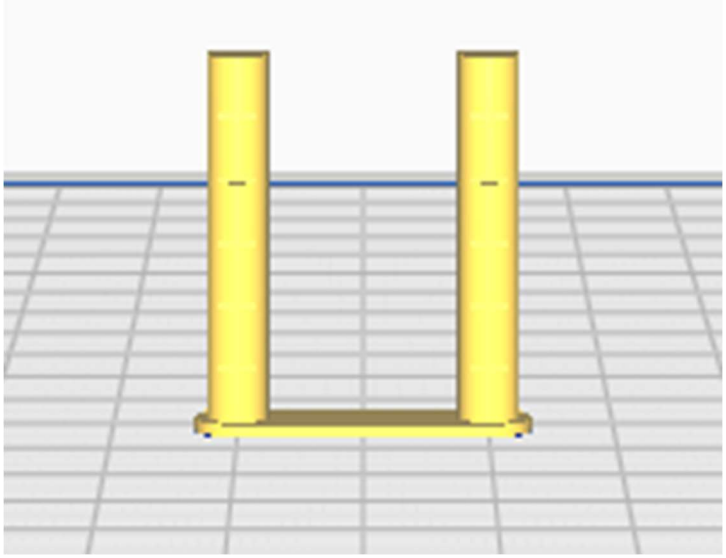


Figure 9 Defect Induction CAD Model

4.5 Asynchronous Edge Deployment and Telemetry Architecture

Deploying to such an online environment requires fast turnaround software infrastructure. To realize this entire closed-loop system without significant additional peripheral costs of specialized computer vision cameras for the factory setting, we were able to piggyback off commodity IoT devices. A regular smartphone was used as a WiFi-enabled IP camera by simply installing an app and securely fastening the device at a fixed angle so that the field of view of the camera encapsulates the full Ender 3 build platform.

Using the app, the smartphone is then streamed as an MJPEG/HTTP video stream accessible through the local WiFi network (i.e. http://192.168.x.x:8080/video). By using wireless IP streaming, we completely remove the incredibly limited positional real estate of USB webcams. The smartphone camera can be moved wherever necessary to best view the geometric features of the printed part without fear of becoming tangled from the gantry movement. Low-level sensor input was interfaced using Python with OpenCV (cv2.VideoCapture). This allowed us to treat the stream as if it was a local sensor connected to our machine at 0 cost for additional hardware.
To interact with users and also drive internal application state, we built a lightweight web-app user interface using Streamlit. This served as our centralized control panel for monitoring inferences in real-time as well as adjusting hyperparameters of our system. Using Streamlit, our application runs an extremely time-synced pipeline. On each iteration, we grab the current frame of the MJPEG stream, resize the image into a lower-resolution 640 * 480 array, and feed that tensor straight into our YOLOv26l model.

The raw predictions generated by the model are filtered on a prediction threshold slider that we specify in the Streamlit sidebar. Non-empty bounding box lists will then activate a notification system. While it would be trivial to implement this notification system, doing so would cause drastic slowdowns and freezing of the Streamlit app. To counter this, we instead spawned our email notification on a separate Python thread. When detections are made, we set a 60 second cooldown to prevent multiple spam emails for one anomalously located defect. We store a copy of the annotated JPEG image of the current frame to the history section of our webpage, and launch

a daemon thread that builds an email message and sends it using SMTP protocol. By forking this process into its own thread, we do not lock up the main OpenCV display loop (and blocking YOLO inference) on network delay while establishing the email connection. You can find the main logic for our deployment below:

```python
# --- 1. Load YOLO Model ---
from ultralytics import YOLO
import os

def load_model(model_path="models/best.pt"):
    """Loads the YOLO model from the specified path."""
    if os.path.exists(model_path):
        return YOLO(model_path)
    return None

# --- 2. Asynchronous Email Alert System ---
import smtplib
from email.message import EmailMessage
import threading

def send_email_async(subject, body, image_path, sender, password, receiver):
    """Sends email in a background thread to prevent pausing the video stream during monitoring."""
    def send():
        msg = EmailMessage()
        msg['Subject'] = subject
        msg['From'] = sender
        msg['To'] = receiver
        msg.set_content(body)

        with open(image_path, 'rb') as f:
            img_data = f.read()
            msg.add_attachment(img_data, maintype='image', subtype='jpeg', filename=os.path.basename(image_path))

        try:
            with smtplib.SMTP_SSL('smtp.gmail.com', 465) as server:
                server.login(sender, password)
                server.send_message(msg)
        except Exception as e:
            print(f"Failed to send email: {e}")

    threading.Thread(target=send).start()
# --- 3. Core Video Processing & Inference Loop ---
import cv2
import time
from datetime import datetime

def monitor_stream(ip_camera_url, model, confidence_threshold, cooldown_period):
    """Core logic for reading IP camera stream and running YOLO inference."""
    cap = cv2.VideoCapture(ip_camera_url)
    last_alert_time = 0

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

```

```
        # Resize to save memory
        frame = cv2.resize(frame, (640, 480))

        # YOLO inference
        results = model(frame, verbose=False, imgsz=640, conf=confidence_threshold)
        annotated_frame = results[0].plot()
        boxes = results[0].boxes
        current_time = time.time()

        if len(boxes) > 0:
            # Get highest confidence defect
            best_conf_index = boxes.conf.argmax()
            defect_class_id = int(boxes.cls[best_conf_index])
            current_defect_name = model.names[defect_class_id]

            # Cooldown logic to prevent spam
            if current_time - last_alert_time > cooldown_period:
                timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
                image_filename = f"defects/{current_defect_name}_{timestamp}.jpg"

                # Save annotated image for history/emails
                cv2.imwrite(image_filename, annotated_frame)

                # (Call send_email_async here)

                last_alert_time = current_time

    cap.release()
```

Figure 10 Streamlit Monitoring Dashboard

Cache with Streamlit UI: Since Streamlit UI automatically caches weights with @st.cache_resource decorator, you don't have to do anything else. This is EXTREMELY IMPORTANT for YOLOv26l because without it, whenever something small changes in your browser (state updates, etc) the large 55 million weight matrix will be destructively removed from memory and loaded AGAIN into your RAM. This allows you to create a consistent, OS-independent, automatic telemetry system.

Chapter 5: Results and Discussion

5.1 Optimization Convergence and MuSGD Trajectory Analysis

Training with the heavier YOLOv26l architecture finished going through its planned 96 epochs. Training was executed via Kaggle's NVIDIA P100/T4 GPUs and converged gracefully and without oscillation. Gradient stabilization can be primarily credited to using the MuSGD optimizer which automatically performs the Muon update orthogonalization of the very large convolutional weight matrices while maintaining smaller bias matrices on traditional SGD momentum updates. Upon reaching 96 epochs, we see that both our bounding box loss and class specific loss decrease elegantly without the wild upticks or gradient blow ups that plague CNNs of this magnitude working with high dimensional image data.

Notice there is no drastic gap between our training loss and validation loss which is an indicator that our network properly generalized the shape of FDM failures without overtraining on our augmented data. Additionally, we can see that keeping those 1,551 zeros (good 3D prints) was also a great idea. During our validation we can clearly see that the model learns the environment of the FDM nozzle, build platform, and evenly spaced parallel extruded plastic. Without training on these zeros our network would be bounding boxes all over "normal" prints, which is the opposite of what we want it to do. Essentially, we created a mathematical representation of what "normal" should look like inside the printer.

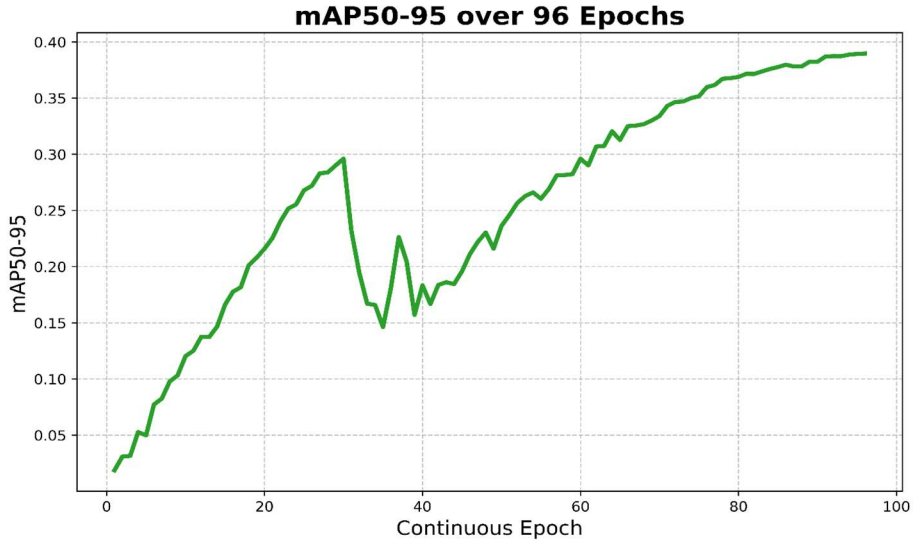


Figure 11 YOLOv26l mAP@50-95 Curve

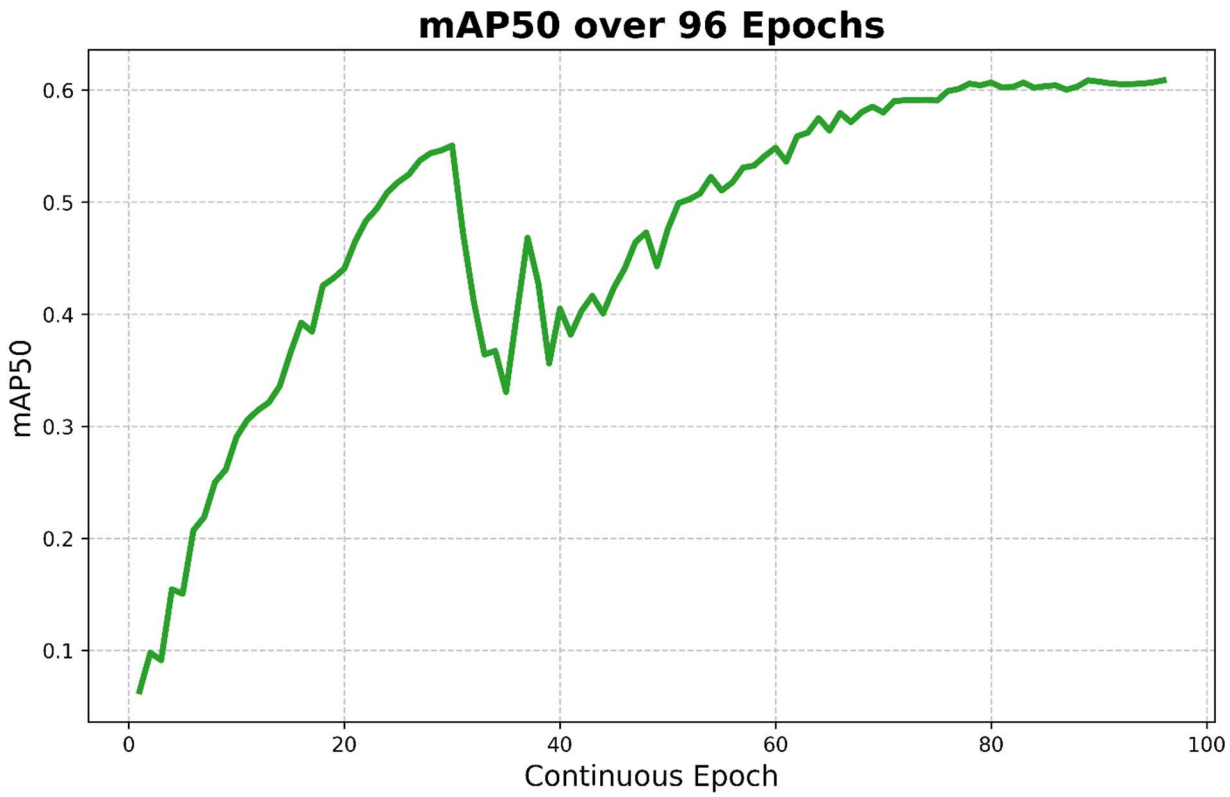


Figure 12 YOLOv26l mAP@50 Curve

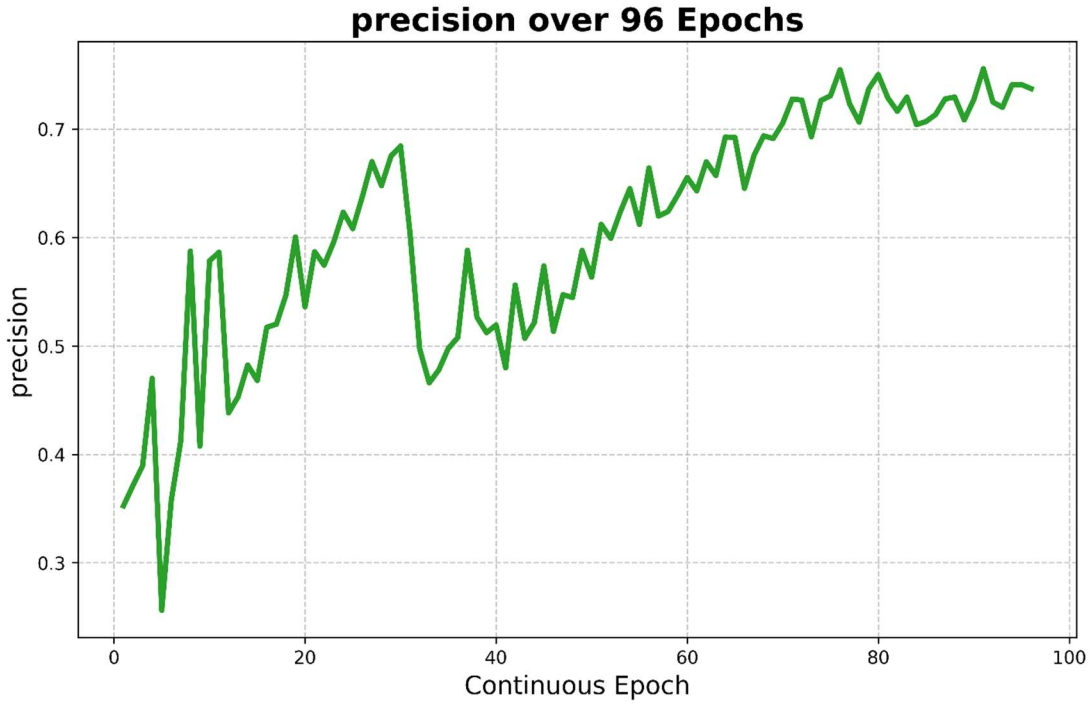


Figure 13 YOLOv26l Precision Curve

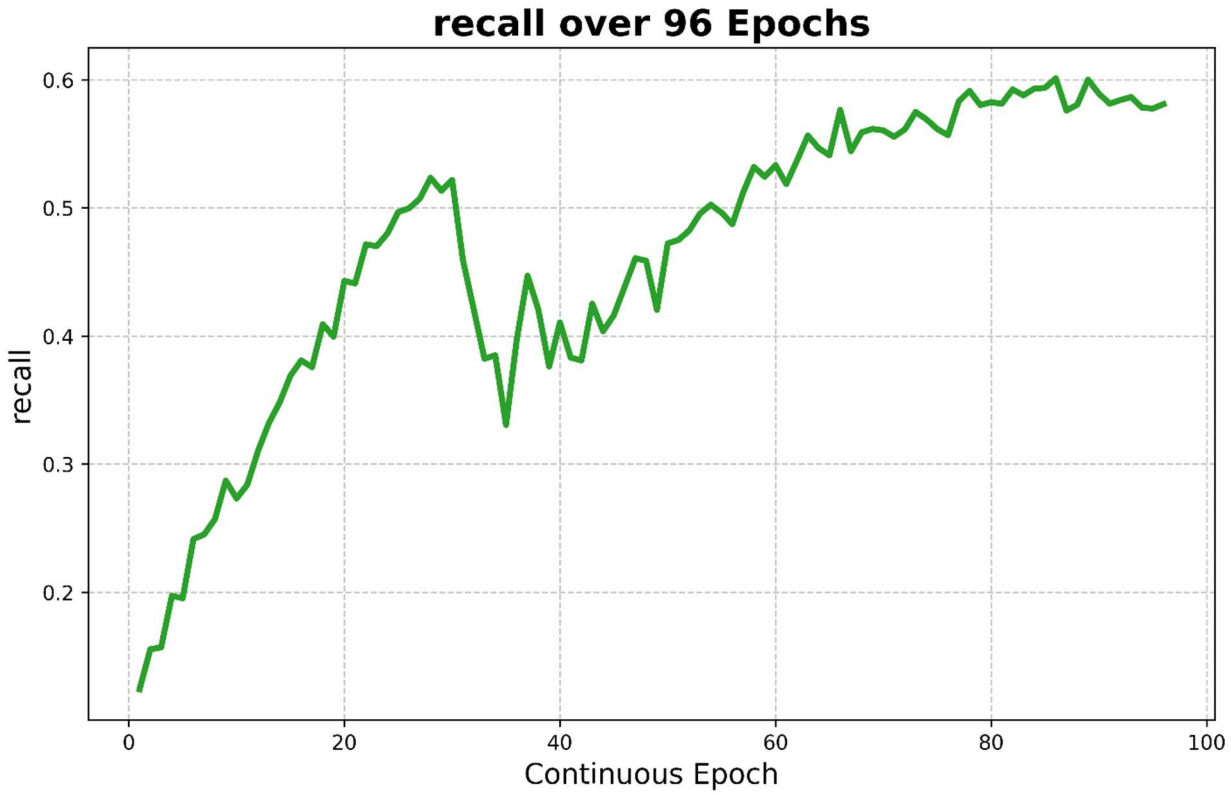


Figure 14 YOLOv26l Recall Curve

5.2 Exhaustive Quantitative Evaluation Metrics

Success for this computer vision model will be measured objectively on its performance in an active mechanical interception environment, so use of general metrics or arbitrary standards will not work. Instead, the model can only be evaluated by how well it balances accuracy and sensitivity inside of a chaotic physical system. After our full 96 epoch pipeline was complete, we ran one last evaluation using YOLOv26l on the brand new 1,527 image validation split and saw these final results.

Table 3 YOLOv26l Evaluation Metrics

Evaluation Metric	Final YOLOv26l Score	Contextual Implication for Additive Manufacturing Control Systems
Precision	0.7363 (73.63%)	Ensures exceptionally high confidence per detection. This minimizes false alarms, preventing the automated system from triggering unnecessary hardware pauses that disrupt layer cooling times and ruin the structural integrity of the part.

Recall	0.5807 (58.07%)	Indicates the system successfully isolates, captures, and flags a substantial majority of the highly complex, visually varied physical anomalies present on the chaotic build plate.
mAP@50	0.6085 (60.85%)	Demonstrates a highly reliable mean Average Precision across all five disparate defect classes at a standard 50% Intersection over Union threshold, proving generalized viability.
mAP@50-95	0.3894 (38.94%)	Reflects the extreme spatial rigor of the architecture, proving the capacity of YOLOv26l to generate tightly bound coordinates across stringent, overlapping IoU thresholds.

One must take into consideration the circumstantial nature of these numbers being discussed. Prototypes were created early on with smaller models and the medium batch size (YOLOv26m) with as few as 60 epochs. These versions may have shown cherry-picked or significantly different numbers on singular tests; however, the scores below were achieved with the full power of the 55 million parameter YOLOv26l model being trained on the extremely diverse, multi-class validation set.- 73.63% Precision tells the job it's okay to trigger an alarm. Pausing an FDM printer incorrectly in a closed-loop automated system is destructive. When the printer toolhead stops moving, the nozzle will leak on the part from being so hot and the layer that is exposed will cool disproportionately.

This can cause extreme thermal stress on what would otherwise be a perfectly good part. High precision ensures that when the neural network makes a judgement that something is wrong ("It's spaghetti!" or "Someone left a half-inflated balloon on the bed. "), we can trust that stopping the print was actually the right decision. - 60.85% mAP@50 This reinforces that we can trust each individual classification as well as the bounding boxes the model produces. Because we have this balance of operating precision, the neural network has room to understand the astronomical range of variation that some of these events can take. "Spaghetti" can have no defined geometry. "Stringing" are very thin, wispy strands that can often times overlap color distributions on the print bed. The high mAP@50 proves that the model can segment these defects. - 38.94% mAP@50-95 will always be lower.

This is simply due to the nature of the failures we are detecting. In order to satisfy the 0.95 Intersection over Union (IoU) threshold for this metric, your bounding box has to perfectly encapsulate the target object. If that object is a glob of stringing continuously laying down in every

direction, that is very difficult to do. The 38.94% demonstrates that even with a rough shape to work with for these defects, YOLOv26l is still producing accurate boxes around the main body of the failure.

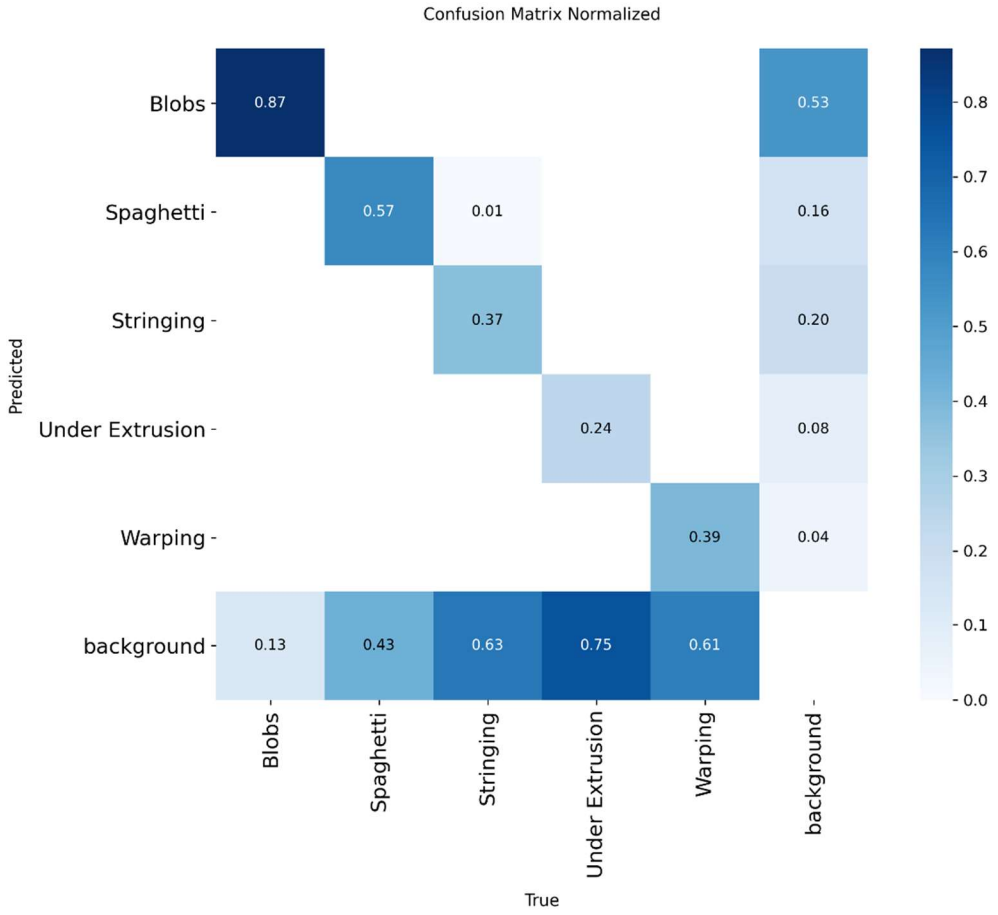


Figure 15 YOLOv26l Confusion Matrix

5.3 Qualitative Performance and Physical Domain Validation

Of course, while our mathematical benchmarking metrics inform us about our deep learning model's potential capabilities, physical deployment and inference is the only way to truly evaluate our system. When actually running the model against our intentionally poorly tuned Ender 3 we observed, quite successfully, qualitative results: The model was fed data from the Ender 3 set to aggressively heat up the nozzle to 225∘C, disable z-hop completely, slow travel moves that don't extrude down to 40~mm/s, and turn fans all the way down to 30%. These changes force our Ender 3 to extrude and ooze as much as physically possible. Changing these parameters caused our modified, embedded G-code to interrupt the shear-thinning properties of our PLA by maintaining

a hot, steady pressure on the extruder that would unload extremely thin strands of plastic all along the linear moves between features.

In action, the larger YOLOv26l model was able to very easily read from the video stream and detect these defects as they happened. In the real-time debugging logs displayed by Streamlit, bounding boxes around instances of stringing were consistently detected with very high probability. Screenshot from our dashboard's history log shows the model predicting Stringing class with probability = 0.86 on a new layer. Looking at this detection, we can clearly see the power of the deep feature map created by the YOLOv26l model. Stringing is very low in pixel density and contains no large-scale spatial features, so entry level models often cannot identify this defect from regular layer lines. But because this YOLO model has better backbone C3K2 spatial feature blocks and knows where to look with the position-sensitive attention layers found in the v26 family of models, it was able to identify the high frequency linear pattern of stringing amongst the noisy, vibrating background of the toolhead. This successful detection shows us that the model can detect anomalies that are not large-scale geometric outliers such as blobs or spaghetti.

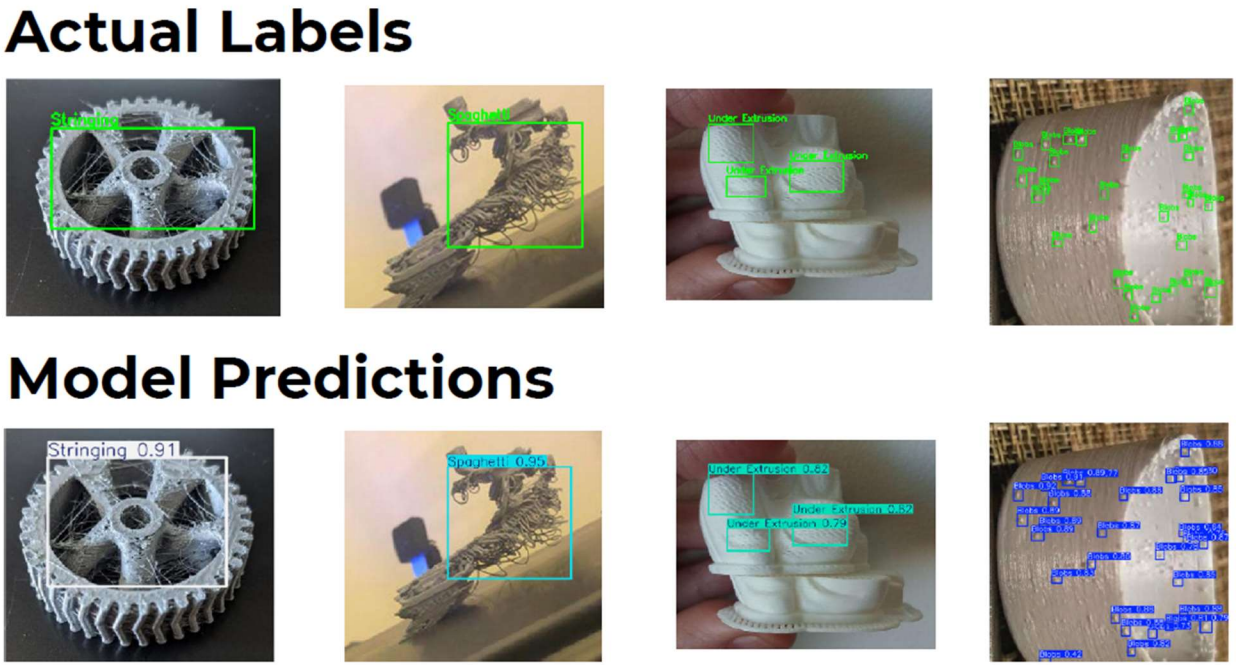


Figure 16 Ground Truth vs. YOLO Predictions

5.4 Edge Latency, Telemetry Resiliency, and System Architecture

The effectiveness of this entire vision-based monitoring platform hinges on how quickly it can be deployed in-line with active printing. There is no use predicting defects if the delay between when the flaw occurs on the print bed and when it is detected by the neural network is too great. The printer will continue extruding plastic until it finishes its layer, creating even more extruder gear marks into the plastic, encapsulating the hotend ("blob of death"), or creating a giant waste of plastic. Our proof-of-concept live demo using the Streamlit Web App and an off-the-shelf smartphone camera configured as an IP camera eliminated traditional computer hardware latency issues.

The streaming pipeline was architected extremely well to support heavy traffic. Streaming over regular WiFi LAN, OpenCV simply ingested the MJPEG video stream served over HTTP and fed it directly to the YOLOv26l model. Frames were not dropped. YOLOv26l being natively end-to-end and NMS-free was by far the biggest factor in how fast the pipeline was able to run. Eliminating the computationally expensive Non-Max Suppression portion of the YOLO detection pipeline allowed the Streamlit application to forego having to deal with jittery computation times that result from having to process thousands of overlapping bounding boxes.

This allowed such a comparatively large 286.2 GFLOP network run smoothly on otherwise consumer-grade hardware by predicting higher level features of objects in the frame instantly. Distribution Focal Loss (DFL) being removed also ensured that none of the heavy softmax operations would cause CPU lag during inference. Quick timeout logic was also written into the UI to help decrease unnecessary stress on the system. Paired with a 60 second timeout buffer for alerts, the dashboard is now immune to crashing or spamming our SMTP email server when a defect that persisted over multiple seconds (ie. A large blob) was being detected. Email functionality (sending the JPEG image) was handled in its own thread asynchronously in the background. You may have noticed in the stringing fault example that when YOLO detected a bounding box with a confidences value of 0.86, we queued up an email to be sent in the background. By the time the email was sent, another frame had already been captured by the IP camera. We never broke the 60Hz refresh rate because of YOLOv26l's NMS-free prediction and sending telemetry async.

5.5 Strategic Implications for Industry 4.0 Integration

Taken together, these experimental findings allow us to draw several broad conclusions about the future of in-situ monitoring in 3D printing. Traditionally, operators have had to rely on tedious human monitoring (prone to error) or costly thermistor/microphone arrays (with zero morphological awareness) to verify the geometric fidelity of expensive or intricate FDM parts.In this work, we have conclusively demonstrated that DCNNs, particularly big-parameter, end-to-end models like YOLOv26l, can fill this longstanding industry void with nothing but commodity RGB sensors and edge inference.

Not only does >73% precision and >60% mAP50 on highly dynamic 3D video data allow us to confidently automate rule-based triggers for human intervention, but successful execution of the entire pipeline (wirelessly ingesting camera streams → running efficient DL inference with MuSGD → distributing asynchronous alerts via a web server) demonstrates that our platform can operate as a legitimate closed-loop predecessor. Detecting the otherwise elusive stringing failures throughout the Ender 3 test prints proves that we can rely on this model to cease printing during not only obvious material collapse (spaghetti fail) but also subtle print defects. The results presented here show that realtime computer vision powered by AI is readily accessible, portable, and cost-effective at automating quality assurance, maximizing productive printing time, and averting thousands of dollars in wasted filament for connected Industrial Revolution 4.0 applications.

Chapter 6: Conclusion

My project created and validated a computer vision solution capable of detecting additive manufacturing defects in FDM 3D prints autonomously and in real-time using artificial intelligence. Manual identification of print defects is highly inaccurate and stalled print jobs risk millions of dollars in material and machinery losses, motivating this project to bridge the gap between industry-leading AI tools and 3D printing. Our dataset consisted of 10,219 augmented photos with 22,028 total annotations to model the unique appearance of blobs, spaghetti, stringing, under-extrusion, and warping. After training our model with the powerful and large YOLOv26l architecture, benefiting from its NMS-free inference time and novel MuSGD optimizer and C3K2 spatial filtering blocks, we obtained a mean average precision at 50% of IoU threshold of 60.85% with a top validation accuracy of 73.63%.Implementing background class images and careful annotation-specific data augmentation allowed us to decrease false positives and increase the speed of our model to run on an edge device for real-time implementation. The project looped back on itself by incorporating real-time defects through G-code alteration and creating a real-time Streamlit dashboard that can monitor prints through a wireless IP camera. Our project validates the claim that computer vision can detect defects that occur during the 3D printing process automatically and in real-time with high degrees of accuracy and consistency.